ABSTRACT

Currently, there is a growing trend which shows that users are moving from text-only prompts to multimodal reasoning, using visual inputs to solve complex problems.
Visual Question Answering (VQA) has emerged as a critical benchmark for evaluating those capabilities of AI systems. However, existing VQA systems suffer from a significant, well-documented challenge known as language bias, where models exploit spurious statistical correlations between question types and frequent answers in the training data, rather than grounding their predictions in actual visual content. This work addresses the problem by proposing X-VQA, an explainable VQA framework trained and evaluated on the GQA dataset, which is constructed from scene graphs to demand genuine multi-step reasoning.

The proposed methodology integrates three pretrained components into a unified architecture: A Swin Transformer (Base) for hierarchal visual feature extraction, an ELECTRA-base model for contextual question encoding and a 6-layer deep Multimodal Cross-Attention Network (MCAN) with feed forward sublayers for bidirectional vision-language fusion. Training was conducted in two phases: an initial frozen-encoder phase to stabilize the fusion layer, followed by end-to-end fine tuning with differential learning rates and cosine warmup scheduling. Explainability was implemented via Score-CAM, a gradient-free activation mapping technique applied to the final Swin transformer block to produce clean, faithful visual evidence for the model's predictions.

The model achieves a final validation accuracy of 55.32% on the GQA balanced split, improving consistently across all five training epochs starting from 47.74% at epoch one. On the GQA-OOD benchmark, the model scores 59.35% on the Head distribution and 39.76% on the Tail distribution. The Head-Tail gap of 19.6 percentage points quantitatively demonstrates the model's reliance on visual features for frequent concept pairings while highlighting residual linguistic bias on rare combinations. Score-CAM heatmaps confirm that the model attends to semantically relevant image regions when forming its predictions.

This project shows that a carefully designed multimodal architecture combined with a principled two-phase training strategy, can achieve competitive reasoning on a challenging out-of-distribution benchmark. The Score-CAM integration provides enhanced interpretability, addressing the trust deficit in black-box VQA systems.
The complete pipeline – from training to inference – is deployed as a Streamlit dashboard, built using PyTorch, HuggingFace Transformers, pytorch-grad-cam library and PIL library.

Table of Contents

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 Overview of the Project

Visual Question Answering (VQA) represents one of the most crucial as well as demanding intersections of computer vision and natural language processing. Given an image and a free-form natural language question about that image, a VQA system must produce a correct, grounded answer – a task that requires complete, simultaneous understanding of visual content, linguistic intent and their relationship.

It needs to bridge the gap between pixels and tokens. Not just see the image, but to reason, understand and converse about its content basic on a specific question. Unlike image classification or object detection, it cannot be solved by either modality alone, the model must reason across both with spatial and detail-oriented precision.

The rapid advancement of transformer-based architectures from matching patterns to matching relationships has fundamentally changed the possibilities in this domain. Vision Transformers like Swin Transformer have demonstrated that hierarchal, shifted-window self-attention can extract richer spatial representations than traditional convolutional neural networks (CNNs). Language models such as ELECTRA have shown that efficient discriminative pretraining produces encoders that understand subtle, nuanced query intent with fewer parameters than traditional masked language models like BERT (Bidirectional Encoder Representation).

This project, X-VQA (Explainable Visual Question Answering) integrates a Swin Transformer visual encoder, an ELECTRA text encoder, and a 6-layer deep Multimodal Cross-Attention Network (MCAN) into a unified architecture trained on the GQA (Geometric Question Answering) dataset. However, it goes beyond prediction accuracy by incorporating Score-CAM (Score-weighted Class Activation Mapping), a gradient free visual explanation method, to produce spatial heatmaps showcasing which regions the model attended to while forming its answer. The complete system is deployed through an interactive Streamlit dashboard.

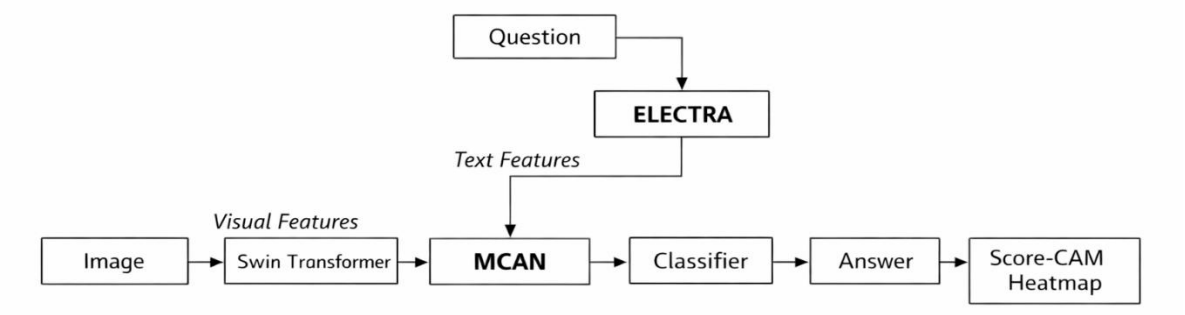


Figure 1.1: High-level System Overview of the Proposed Framework

1.2 Purpose and Objectives

Despite achieving impressive benchmark numbers, the majority of VQA models are untrustworthy. This can be explained through the “Clever Hans” effect where a horse appeared to be doing math but was just reading the trainer’s body language.

A well-documented phenomenon called language bias causes models to answer question by exploiting statistical regularities in the training data rather than by examining the image.

For example, if the question is “What is the colour of the banana?”, a biased model will answer “yellow” not because it looked at the image, but because yellow is the most frequent answer to banana-related questions in training data. It would give the same answer even if the image showed a green banana, red apple or no fruit at all.

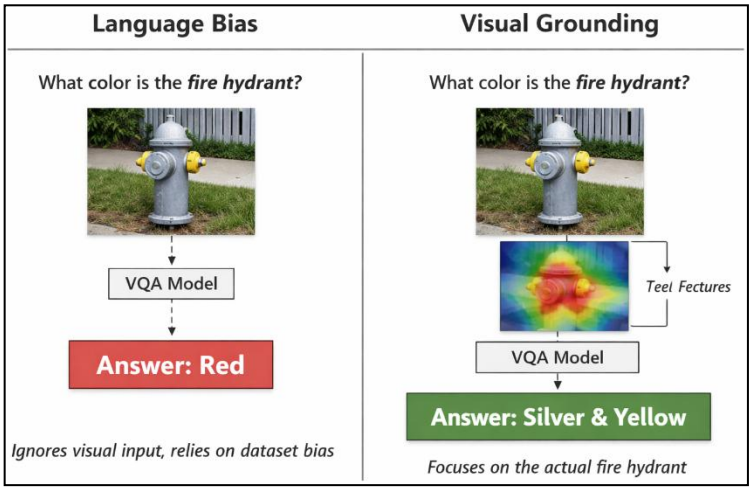


Figure 1.2: Illustration of Language Bias in Visual Question Answering

This is a major issue, a medical imaging system that answers questions about X-rays based on linguistic priors rather than visual evidence is dangerous. An autonomous system that ignores visual context in favour of statistical shortcuts will perform poorly in the novel, out-of-distribution scenarios that matter most.

The GQA-OOD benchmark was designed to expose this exact failure. In a standard GQA evaluation, a model can fake a high score, however here it evaluates the model on rare, infrequent, counterintuitive concept pairings where linguistic shortcuts provide no advantage and genuine visual reasoning is the only path to a correct answer.

Still, even when a VQA model answers correctly, there is no guarantee that it did so for the right reasons. Without a mechanism to inspect the model’s reasoning, it is impossible to distinguish visual understanding from sophisticated pattern matching.

Score-CAM, an explainable AI (XAI) technique addresses this by producing gradient-free, spatial attribution maps showing not just what the model answered, but where in the image it looked to arrive at the answer. Essentially, it helps in visualizing the reasoning and internal decision-making process.

The specific objectives of this project are :-

- Hierarchal Visual Featurization
  To implement a Swin Transformer (Base) as the visual encoder, leveraging its shifted window attention mechanism to capture multiscale spatial features that outperform standard CNNs on complex scene understanding tasks

- Contextual Question Encoding
  To utilize ELECTRA-base as the text encoder, exploiting its replaced token detection pretraining objective to produce richer contextual representations of complex natural language queries with greater parameter efficiency than BERT-based alternatives.

- Multimodal Reasoning via Deep MCAN
  To develop a 6-layer deep Cross-Attention Multimodal Network with feed-forward sublayers, enabling bidirectional co-attention where visual patches attend to question tokens and question tokens attend to visual patches simultaneously across multiple layers of abstraction.

- Explainable Visual Grounding
  To implement Score-cam on the final Swin Transformer block, producing gradient-free, class-specific spatial heatmaps that visually ground the model's predictions and provide interpretable evidence of its reasoning process.

- Bias Quantification via OOD Evaluation
  To evaluate the trained model on the GQA-OOD benchmark’s Head and Tail distributions, quantitatively measuring the degree of language bias and demonstrating the model's reliance on visual features for rare, concept pairings.

- Interactive Dashboard Deployment
  To build a Streamlit dashboard that accepts user-uploaded images and natural language questions, displays the predicted answer along with confidence score, and renders the Score-CAM heatmap alongside the original image.

1.3 Problem Definition

Formally, the VQA problem is defined as follows. Given an Image I and a natural language question Q, the goal is to predict an answer â from a closed vocabulary A of N possible answers, such that:

$\hat{a} = \arg\max_{a \in A} P(a|I,Q;\theta)$

where θ represents the learned parameters of the model.

The specific problems this project addresses are threefold.

First, standard VQA datasets such as VQAv2 contain significant language bias – models trained on them can achieve hallucinated competitive accuracy without learning genuine visual-linguistic alignment. This project uses the GQA dataset, constructed from Visual Genome scene graphs, which enforces compositional, multi-step reasoning and eliminates the trivially guessable questions that inflate performance on biased datasets.

Second, the fusion of visual and linguistic modalities is non-trivial. Simple concatenation or element-wise operations fail to capture the rich relational structure between image regions and question words. This project addresses this through a 6-layer MCAN that performs iterative, bidirectional cross-attention, allowing each modality to progressively refine its representation by attending to the other.

Third, the opacity of neural VQA models undermines trust in their predictions. This project addresses this through Score-CAM integration, which produces spatial attribution maps directly from the Swin Transformer's activation space without requiring gradient computation, resulting in cleaner, noise-free, stable and faithful explanations than gradient-based alternatives.

Property	VQA v2	GQA
Language Bias	High	Low (scene-graph grounded)
Multi-Step Reasoning	No	Yes
OOD Evaluation Split	No	Yes (Head/Tail)
Answer Vocabulary Size	~3,129	1,833
Questions Require Visual Grounding	Partial	Enforced
Dataset Construction	Natural images + crowd-sourced Q/A	Scene-graph derived questions (synthetic but semantically grounded)
Question Diversity	Broad but shallow	Structured, compositional, multi-hop
Evaluation Focus	Accuracy Only	Accuracy + robustness (bias, compositionality, grounding)

Table 1: Comparison of problem characteristics across VQA datasets
The table highlights key differences between VQA v2 and GQA, including language bias, reasoning requirements, evaluation splits, answer vocabulary size, and the extent of visual grounding. Unlike VQA v2, GQA enforces multi-step reasoning and scene-graph grounding, making it a more robust benchmark for evaluating compositional and out-of-distribution performance.

CHAPTER 2
BACKGROUND THEORY AND LITERATURE REVIEW

2.1 Evolution of Visual Question Answering

The VQA task was formally introduced by Antol et al. [1] with the VQA v1 dataset, which paired MS-COCO images with open ended human-generated questions. It was a vision to move beyond simple object recognition and create something that required a "higher-level" understanding of scenes.

Early approaches followed a straightforward pipeline : extract image features using a pretrained CNN (usually VGGNet or ResNet), encode the question using an LSTM (Long Short-Term Memory), combine the two representations through element-wise multiplication or concatenation and classify over a fixed answer vocabulary.
While these models achieved reasonable accuracy on standard analysis, subsequent analysis uncovered a fundamental flaw – they are answering questions without meaningfully looking at the images.

Goyal et al. [2] exposed this problem systematically by constructing VQA v2, where each question is paired with two images that yield different correct answers. Models that relied on language priors failed dramatically on this balanced split, demonstrating that high VQA v1 accuracy largely stemmed from dataset bias rather than genuine visual reasoning.
This finding motivated a shift towards architectures that enforce stronger visual-linguistic interaction. The introduction of attention mechanisms was a major breakthrough.

Yang et al. [3] proposed Stacked Attention Networks (SAN) which iteratively refined the image region attended to based on the question. Anderson et al. [4] introduced Bottom-Up and Top-Down attention (UpDn), using faster R-CNN to extract salient object region features rather than a uniform spatial grid, achieving groundbreaking results at the time and formed the basis of the winning entry in the 2017 VQA challenge.

The introduction of Transformers, first proposed in the seminal Attention Is All You Need paper [5], further elevated VQA performance. Models including ViLBERT [6], LXMERT [7] and Oscar [8] demonstrated that large-scale vision-language pretraining on hundreds of millions of image-caption pairs could produce powerful multimodal representations. However, these models are computationally prohibitive, require massive pretraining corpora, and remain largely opaque in their reasoning.

X-VQA addresses the same core problem – multimodal reasoning with bias awareness but from the perspective of architectural transparency, trust, and explainability rather that scale.

Model	Year	Visual Encoder	Text Encoder	Fusion	GQA Accuracy
CNN-LSTM Baseline	2015	ResNet	LSTM	Concat	~35%
Bottom-Up Attention	2018	Faster R-CNN	GRU	Top-Down Attn	~49%
LXMERT	2019	ResNet + ROI	BERT	Cross-Attn	~60%
Oscar	2020	Faster R-CNN	BERT	Tag-guided	~61%
X-VQA (this work)	2026	Swin-B	ELECTRA	6-layer MCAN	55.32%

Table 2: Evolution of VQA Architectures highlighting key innovations over the years

2.2 Visual Encoder

The Swin Transformer [9] is a hierarchal vision transformer that computes self-attention within local shifted windows rather than globally across all patches. This design produces multi-scale feature maps analogous to a CNN feature pyramid, making it significantly more effective than standard ViT [10] for spatially grounded tasks.

Swin-Base achieves 83.5% top-1 accuracy on ImageNet with 88M parameters. It was selected over ResNet-based encoders [11] due to its superior scene understanding capability, and over plain ViT due to its hierarchical structure which preserves spatial locality essential for visual grounding via Score-CAM.

2.3 Text Encoder

ELECTRA [12] replaces BERT's [13] masked language modelling objective with a replaced token detection task, where a discriminator identifies which tokens in a sequence were replaced by a small generator network. Since, every token participates in training rather than just the masked 15%, ELECTRA learns from a substantially denser training signal and achieves BERT-large level performance at BERT-base parameter count.

It was selected over BERT for its superior efficiency and over GPT-style decoders because question encoding requires bidirectional context — the meaning of a word in a VQA question depends on both what precedes and follows it.

2.4 Multimodal Fusion

The Deep Modular Co-Attention Network [14] performs bidirectional cross-attention between visual and linguistic features across multiple stacked layers, allowing each modality to dynamically attend to the other.

Unlike simple concatenation or element-wise fusion, MCAN builds progressively abstract relational representations – early layers align noun-region correspondences while deeper layers capture complex spatial and relational predicates.

The original paper demonstrated that a 6-layer configuration significantly outperforms shallower variants on compositional reasoning benchmarks, which motivated the use of the full depth in this project.

2.5 Explainability

Score-CAM [15] is a gradient-free activation mapping method that weights each feature map by its causal contribution to the model's confidence score, measured by masking the input with that activation and observing the change in output.

Unlike GradCAM [16], it does not rely on backpropagated gradients, making it more stable and faithful for transformer-based architectures where gradient flow through attention layers is difficult to interpret reliably. It was selected over GradCAM and attention rollout specifically because of this gradient independence.

2.6 Dataset Used

Hudson and Manning [17] introduced GQA by generating questions programmatically from Visual Genome scene graphs, ensuring every question has a unique, unambiguous, visually grounded answer requiring genuine multi-step reasoning. Unlike VQA v2 where models can exploit linguistic priors to achieve high accuracy without visual understanding, GQA's compositional structure enforces that correct answers require actual scene comprehension.

The GQA-OOD benchmark [18] extends this by splitting the validation set into a Head distribution – questions whose answer matches frequent statistical associations and a Tail distribution – questions whose answer is rare or contradicts common priors.

The Head-Tail accuracy gap directly quantifies language bias: a model that reasons visually will show a smaller gap than one relying on priors. This benchmark forms the primary bias evaluation protocol for this project.

2.7 Limitations of Existing Systems

Despite significant progress, existing VQA systems exhibit several limitations that motivate this work. First, large pretrained multimodal models (ViLBERT, Oscar, CLIP-based VQA) require massive pretraining on hundreds of millions of image-text pairs, making them inaccessible for resource-constrained research settings and difficult to audit for bias.

Gradient-based explainability methods produce unstable explanations for transformer architectures, undermining trust in the visual groundings they provide.

Very few VQA systems are evaluated on OOD benchmarks. Most report only in-distribution accuracy, which masks the degree of language bias in their predictions. X-VQA directly addresses all three of these limitations through its choice of architecture, explainability method, and evaluation protocol.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Functional Requirements

The X-VQA system must accomplish the following:-

1) Image Ingestion

The system must accept RGB images in standard formats (JPEG, PNG) and preprocess them to 224×224 resolution with ImageNet normalization for input to the Swin Transformer.

2) Question Ingestion

The system must accept free-form natural language questions, tokenize them using the ELECTRA tokenizer, and pad or truncate to a maximum length of 40 tokens.

3) Multimodal Inference

The system must perform a forward pass through the full pipeline – Swin encoder, ELECTRA encoder, MCAN fusion, and classifier to produce a probability distribution over 1,833 answer classes.

4) Answer Prediction

The system must return the highest-probability answer from the vocabulary as the predicted answer, along with a confidence score derived from the softmax output.

5) Visual Explanation

The system must generate a Score-CAM heatmap by probing the final Swin Transformer block's activations and overlay it on the input image to show which spatial regions contributed most to the prediction.

6) Dashboard Interface

The system must provide a web-based interface through which a user can upload an image, enter a question, trigger inference, and view the answer, confidence score, original image, and Score-CAM heatmap side by side.

7) Model Persistence

The system must support saving and loading trained model weights, enabling inference without retraining.

8) OOD Evaluation

The system must support batch evaluation over the GQA-OOD Head and Tail splits, reporting per-split accuracy and skipped sample counts.

3.2 Non-Functional Requirements

The attributes the system must exhibit are:-

1) Accuracy

The model must achieve validation accuracy competitive with similarly sized models trained without large-scale pretraining on vision-language corpora. Target: above 50% on GQA balanced validation split.

2) Explainability

Score-CAM heatmaps must be visually coherent — highlighting semantically relevant image regions rather than producing uniform or random attribution patterns.

3) Reproducibility

All random seeds, vocabulary construction, and dataset splits must be deterministic, ensuring that results can be reproduced from the same checkpoint and configuration.

4) Usability

The Streamlit dashboard must require no technical knowledge to operate — a user should be able to obtain an answer and heatmap within 60 seconds of uploading an image.

5) Modularity

The architecture must be implemented as separable, independently testable components (encoders, fusion layer, classifier, CAM wrapper) to support future modification or replacement of individual modules.

6) Portability

The inference pipeline must run on CPU in the absence of a GPU, ensuring the Streamlit dashboard functions on standard development hardware.

7) Scalability of Evaluation

The OOD evaluation pipeline must handle the full GQA-OOD set of 51,045 questions without memory overflow, processing samples individually rather than in large batches.

3.3 Feasibility Study

Technical Feasibility

All components of the proposed system are implemented using mature, well-supported open-source frameworks. PyTorch [1] provides the deep learning backbone. HuggingFace Transformers [2] provides pretrained Swin-Base and ELECTRA-base weights with standardized APIs. The pytorch-grad-cam library provides a tested Score-CAM implementation compatible with transformer architectures.

Streamlit provides rapid dashboard development without frontend engineering.
The GQA dataset and GQA-OOD benchmark are publicly available from Stanford and GitHub respectively. No proprietary tools, licensed datasets, or specialized hardware beyond a single GPU are required.

Economic Feasibility
The entire project was executed on Google Colab using an A100 GPU instance, consuming approximately 40 compute units for the complete 5-epoch training run at a cost of less than ₹500. All software dependencies are open-source and free. The Streamlit dashboard runs locally without cloud hosting costs. The project is therefore economically feasible for an individual academic researcher.

Operational Feasibility
The trained model is packaged as two files — best_model.pth containing the model weights (~922 MB) and answer_to_id.json containing the answer vocabulary (~30 KB).
The Streamlit dashboard loads these files once at startup and serves subsequent inference requests without reloading. Any user with a Python environment and these two files can operate the complete inference system. The system requires no database, no internet connection at inference time, and no special configuration beyond standard Python package installation.

Dimension	Assessment	Justification
Technical	Feasible	All components open-source, well-documented
Economic	Feasible	< ₹500 total compute cost, free software
Operational	Feasible	Two-file deployment, no infrastructure needed
Time	Feasible	Full training completes in ~8 hours on A100

Table 3: Feasibility Summary
This table evaluates the proposed system across technical, economic, operational, and time dimensions. Each dimension is assessed as feasible, with justifications ranging from open-source availability and low compute cost to simple deployment requirements and manageable training time.

CHAPTER 4
PROPOSED METHODOLOGY

This chapter presents the proposed methodology, detailing the overall flow of the work, the model architecture and parameters, and the dataset employed.

4.1 Overview of the Proposed Framework

X-VQA is a classification-based Visual Question Answering framework that takes an image and a natural language question as input and produces a predicted answer from a closed vocabulary of 1,833 unique answers derived from the GQA training set.

The framework follows the established convention in VQA literature — answer prediction is formulated as multi-class classification rather than open-ended generation, consistent with seminal works including MCAN, LXMERT and Oscar . The intelligence of the system lies entirely in the quality of the cross-modal representations learned by the fusion network, not in the size of the output vocabulary.

The pipeline consists of five stages: visual feature extraction via Swin Transformer, question encoding via ELECTRA, dimensionality projection, multimodal fusion via 6-layer MCAN, and answer classification. A parallel explainability pipeline applies Score-CAM to the final Swin block to produce spatial attribution heatmaps at inference time.

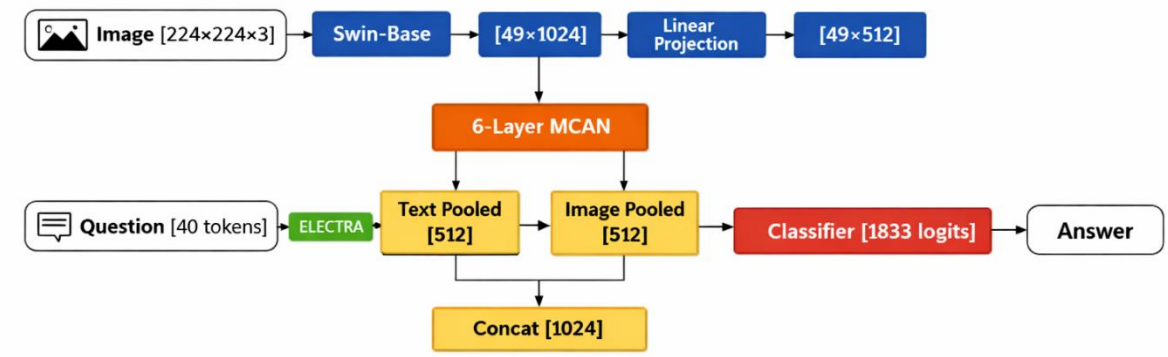


Figure 4.1: End-to-End VQA Pipeline

The architecture combines Swin-Base for image encoding and ELECTRA for question encoding. Although the diagram omits the intermediate [40×768] → Linear Projection → [40×512] step for brevity, this transformation is applied before fusion in the 6-layer MCAN. The concatenated pooled features are classified into 1833 logits to produce the final answer

4.2 Dataset Description

4.2.1 GQA Training Dataset

The GQA dataset [3] was used for training and validation. GQA questions are generated programmatically from Visual Genome scene graphs, ensuring compositional, multi-step spatial reasoning is required for every question. The balanced training split contains 943,000 questions across approximately 72,000 images. The balanced validation split contains 132,062 questions used for per-epoch accuracy evaluation.

Images were downloaded directly from Stanford's servers to Colab local disk during setup. Questions were stored in JSON format with the following relevant fields per question entry: question (string), answer (string), imageId (string identifying the corresponding image file).

4.2.2 Answer Vocabulary Construction

The answer vocabulary was constructed by extracting all unique ground-truth answers from the training set, sorting them alphabetically for deterministic label assignment, and mapping each answer string to an integer index. This produced a vocabulary of 1,833 unique answers, saved as answer_to_id.json for use at both training and inference time. The reverse mapping (integer index to answer string) is derived at runtime for prediction decoding.

4.2.3 GQA-OOD Evaluation Dataset

The GQA-OOD benchmark [4] was used for post-training bias evaluation.

The Head split (33,882 questions) contains questions whose correct answer is statistically frequent for that question type. The Tail split (17,163 questions) contains questions whose correct answer is rare or contradicts common statistical associations. These were loaded directly from ood_val_head.json and ood_val_tail.json in the GQA-OOD repository.

Split	Questions	Images	Purpose
GQA Train (balanced)	943,000	~72,000	Model training
GQA Val (balanced)	132,062	~10,000	Per-epoch validation
GQA-OOD Head	33,882	Subset of val	Bias evaluation
GQA-OOD Tail	17,163	Subset of val	Bias evaluation

Table 4.1: Dataset Statistics

The table summarizes the composition of the GQA dataset used in this project, detailing the number of questions and images across training, validation, and out-of-distribution (OOD) splits. It highlights how each subset supports model training, validation, and bias evaluation.

4.3 Model Architecture

4.3.1 Visual Encoder – Swin Transformer

The visual encoder is microsoft/swin-base-patch4-window7-224 [5], loaded from HuggingFace with pretrained ImageNet weights. It accepts 224×224 RGB images and processes them through four stages of hierarchical window-based self-attention.

The encoder configuration is patch size 4, window size 7×7, producing a final hidden state of shape [batch, 49, 1024] — a sequence of 49 spatial tokens each at 1024 dimensions. The last_hidden_state output is used as the visual feature sequence.

The classification head weights present in the pretrained checkpoint are unused and correctly flagged as UNEXPECTED by the loader — this is expected behaviour when loading an encoder-only backbone.

A linear projection layer maps the 1024-dimensional visual tokens to the 512-dimensional fusion space: vision_proj: Linear(1024 → 512)

4.3.2 Text Encoder – ELECTRA

The text encoder is google/electra-base-discriminator [6], loaded from HuggingFace with pretrained weights. Questions are tokenized using the corresponding ELECTRA tokenizer with padding to a maximum length of 40 tokens and truncation for longer inputs. The encoder produces a contextual hidden state of shape [batch, 40, 768] — one 768-dimensional vector per token. The last_hidden_state output is used as the question feature sequence.

A linear projection layer maps the 768-dimensional text tokens to the 512-dimensional fusion space: text_proj: Linear(768 → 512)

4.3.3 Deep MCAN Fusion

The fusion module is a 6-layer deep Multimodal Cross-Attention Network. Each layer (MCANLayer) contains the following components in order:

- Text-to-Image Cross-Attention: MultiheadAttention(query=text, key=image, value=image, heads=8) with residual connection and LayerNorm
- Image-to-Text Cross-Attention: MultiheadAttention(query=image, key=text, value=text, heads=8) with residual connection and LayerNorm
- Text FFN: Linear(512→2048) → GELU → Dropout(0.1) → Linear(2048→512) with residual connection and LayerNorm
- Image FFN: Linear(512→2048) → GELU → Dropout(0.1) → Linear(2048→512) with residual connection and LayerNorm

Six such layers are stacked sequentially. At each layer, both the text features [batch, 40, 512] and image features [batch, 49, 512] are updated. After all six layers, mean pooling is applied independently to each modality:

text_pooled = mean(text_fused, dim=1) → [batch, 512] image_pooled = mean(image_fused, dim=1) → [batch, 512]

The two pooled representations are concatenated to form a joint representation: combined = concat(text_pooled, image_pooled) → [batch, 1024]

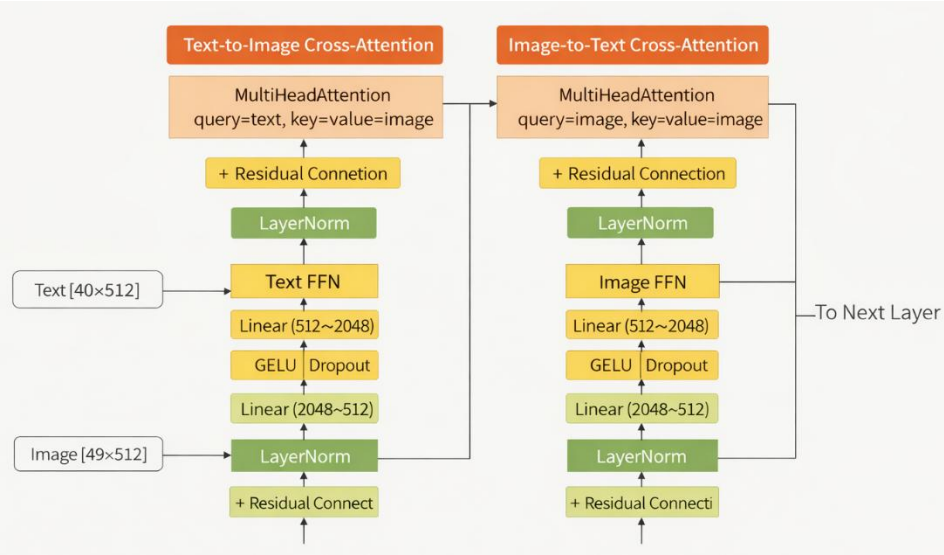


Figure 4.2:
Single MCAN Layer Internals
This diagram shows the structure of a single Multimodal Cross-Attention Network (MCAN) layer, where text and image features interact through bidirectional cross-attention, feed-forward networks, and residual connections before passing to the next layer.

4.3.4 Answer Classifier

The classifier maps the 1024-dimensional joint representation to logits over the answer vocabulary:

Linear(1024 → 512) → GELU → Dropout(0.3) → Linear(512 → 1833)

During training, CrossEntropyLoss is applied to the raw logits. During inference, softmax is applied to obtain a probability distribution, and the argmax gives the predicted answer index.

Component	Module	Input Dim	Output Dim	Parameters
Vision Encoder	Swin-Base	224x224x3	49x1024	~88M
Text Encoder	ELECTRA-base	40 tokens	40x768	~110M
Vision Projection	Linear	1024	512	0.5M
Text Projection	Linear	768	512	0.4M
MCAN (6 layers)	MCANLayer x 6	512	512	~75M
Classifier	MLP	1024	1833	~2M
Total				~276M

Table 4.2: Model Architecture Summary
This table presents the detailed configuration of the proposed multimodal model, outlining each component's module type, input–output dimensions, and parameter count. It highlights the integration of Swin-Base and ELECTRA encoders with MCAN fusion and a classifier, totalling approximately 276 million parameters

4.4 Training Strategy

4.4.1 Two Phase Training

Training was conducted in two phases to prevent catastrophic forgetting of pretrained encoder weights during early training when the randomly initialised MCAN and classifier produce large, unstable gradients.

Phase 1 — Frozen Encoders (Epochs 1–3)
Both the Swin Transformer and ELECTRA encoder parameters were frozen. Only the vision projection, text projection, MCAN, and classifier were trained. This allowed the fusion layers to stabilise their representations before the encoders were exposed to task-specific gradients. The AdamW optimizer was used with a uniform learning rate of 1e-4 and weight decay of 0.01. A cosine schedule with linear warmup over 10% of Phase 1 steps was applied.
Phase 2 — Full Fine-tuning (Epochs 4–5)
All parameters were unfrozen. Differential learning rates were applied to prevent the pretrained encoders from being updated too aggressively relative to the fusion layers, which had already learned task-specific representations in Phase 1.

Component	Learning Rate	Rationale
Swin Encoder	5e-6	Preserve pretrained visual representations
Electra Encoder	5e-6	Preserve pretrained language representations
Vision Projection	1e-5	Moderate update
Text Projection	1e-5	Moderate update
MCAN (all layers)	1e-5	Moderate update
Classifier	1e-5	Moderate Update

Table 4.3: Differential Learning Rates in Phase 2

A cosine warmup schedule over 10% of Phase 2 steps was applied with weight decay 0.01.

4.4.2 Gradient Clipping

Gradient clipping with max norm 1.0 was applied after every backward pass across both phases. This prevents exploding gradients during Phase 2 when the full model including large pretrained encoders participates in backpropagation.

4.4.3 Training Configuration

This section outlines the key hyperparameters used during model training. It specifies optimization settings, learning rate schedules, batch sizes, and regularization strategies that ensure stable convergence and reproducible performance across training phases.

Hyperparameter	Phase 1	Phase 2
Epochs	3	2
Batch Size	32	32
Optimizer	AdamW	AdamW
Learning Rate	1e-4 (uniform)	Differential (Table 6)
Weight Decay	0.01	0.01
LR Schedule	Cosine+Warmup	Cosine+Warmup
Warmup Steps	10% of phase steps	10% of phase steps
Gradient Clipping	1.0	1.0
Hardware	A100 GPU	A100 GPU

Table 4.4: Overview of the Training Hyperparameters

4.5 Score-CAM Visual Explanation

At inference time, Score-CAM is applied to produce a spatial attribution heatmap alongside every prediction. The implementation addresses two key challenges specific to applying Score-CAM to a multimodal transformer.

Challenge 1 – Multimodal Input

Score-CAM was designed for single-input (image-only) models. In X-VQA, the model requires both image and question as input.

This is resolved through a wrapper class (XVQACamWrapper) that holds the tokenized question fixed and exposes only the image as the variable input to Score-CAM's probing mechanism.

Challenge 2 – Batch Size Expansion

Score-CAM internally generates masked versions of the input image and processes them in a batch of up to 64 samples per forward pass. The question tensors must be expanded to match this dynamic batch size. The wrapper's forward method expands input_ids and attention_mask to match pixel_values.shape[0] at each call, resolving the shape mismatch that would otherwise occur in the MCAN attention layers.

Target Layer

Score-CAM hooks are registered on

model.vision_encoder.encoder.layers[-1].blocks[-1].layernorm_before

–the LayerNorm preceding the final Swin attention block.

This layer produces a sequence of shape [batch, 49, 1024].

Reshape Transform

Since Score-CAM expects 2D spatial feature maps [batch, channels, H, W] but Swin produces sequential tokens [batch, 49, 1024], a custom reshape transform converts the sequence to [batch, 1024, 7, 7] by reshaping the 49-length sequence into a 7×7 grid and transposing channel dimensions. Score-CAM then upsamples these 7×7 maps to 224×224 for overlay on the original image.

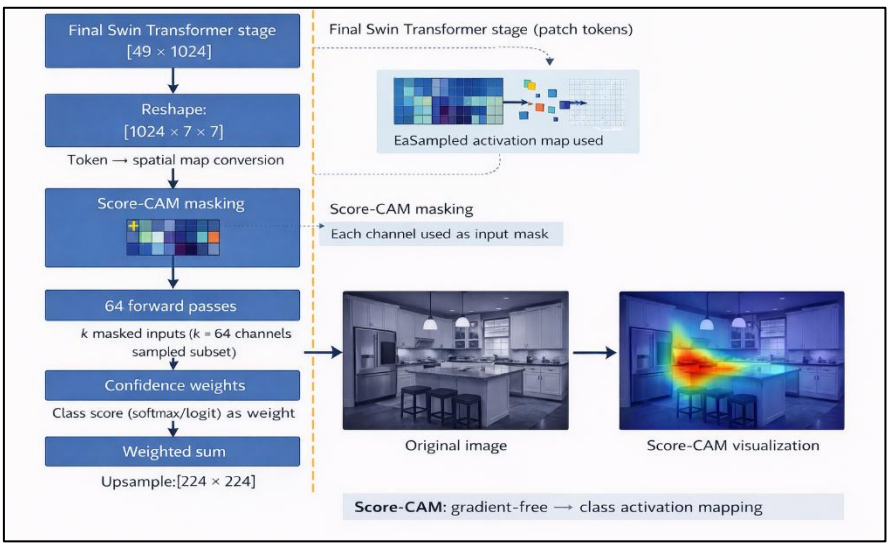


Figure 4.3: Score-CAM applied to a Swin Transformer, illustrating the transformation of patch tokens into spatial feature maps, channel-wise masking with 64 forward passes, and gradient-free aggregation to generate a class activation map over the input image.

4.6 Inference Pipeline

At inference time, the following sequence is executed :-

1. Input image is loaded and converted to RGB.
2. Image processor resizes and normalizes to 224×224, producing pixel_values tensor
3. Question is tokenized to max length 40, producing input_ids and attention_mask
4. Forward pass-through full model produces logits [1,833]
5. Softmax gives confidence scores; argmax gives predicted answer index
6. Answer index is mapped to answer string via reverse vocabulary lookup
7. Score-CAM is applied using the fixed question tokens and variable image input
8. Heatmap is overlaid on the resized original image using show_cam_on_image
9. Answer, confidence score, and heatmap are returned for display

CHAPTER 5
IMPLEMENTATION

5.1 Development Environment

The experiments were conducted using a hybrid setup that combined cloud-based training with local inference. This section details the hardware, software, and supporting libraries employed, ensuring transparency and reproducibility of the development workflow.

Component	Specification
Training Platform	Google Colab (A100 GPU, 40GB VRAM)
Inference Platform	Local Machine (CPU), PyCharm IDE
Python Version	3.12
Deep Learning Framework	PyTorch 2.10.0 + CUDA 12.8
Transformer Library	HuggingFace Transformers
Explainability Library	pytorch-grad-cam 1.5.5
Dashboard Framework	Streamlit
Dataset Storage	Google Drive (mounted in Colab)

Table 5: Overview of the Development Environment

5.2 Data Pipeline Implementation

The GQA Dataset implements PyTorch's Dataset interface. On initialization, it loads the full questions JSON into memory, builds the list of question ID's, and instantiates the Swin image processor and ELECTRA tokenizer. The __getitem__ method performs the following for each sample :-

- Retrieves question text, ground truth answer, and image ID from the questions dictionary

- Constructs the image path, checking both images/images/ and images/ subdirectory structures to handle GQA's zip extraction format
- Opens the image with PIL[7], converts to RGB, and falls back to a blank 224×224 image for corrupted files
- Applies the image processor to produce normalized pixel values
- Tokenizes the question to max length 40 with padding and truncation
- Maps the answer string to its integer label via the vocabulary

Data Loaders were configured with batch size 32, 4 worker processes, and pinned memory for efficient GPU transfer.

5.3 Model Implementation

The full model is implemented across three classes:

1. MCANLayer implements one co-attention layer with two nn.MultiheadAttention modules (8 heads, 512 dimensions), two FFN sublayers (hidden dimension 2048, GELU activation, 0.1 dropout), and four LayerNorm modules with residual connections.
2. MCAN stacks 6 MCANLayer instances in an nn.ModuleList and applies them sequentially, passing updated text and image features from each layer to the next.
3. XVQAModel composes the full pipeline: Swin encoder, ELECTRA encoder, vision and text projection linears, MCAN, and the two-layer classifier with GELU activation and 0.3 dropout.

5.4 Training Implementation

The training loop implements the two-phase strategy as follows.

At epoch 0, get_phase1_optimizer freezes all encoder parameters using requires_grad = False and creates an AdamW optimizer over the remaining trainable parameters.

At the start of epoch 3 (start of Phase 2), get_phase2_optimizer unfreezes all parameters and creates a new AdamW optimizer with the differential learning rate parameter groups defined in Table 4.3.

Each training step follows:

zero gradients → forward pass → CrossEntropyLoss → backward → gradient clip (max norm 1.0) → optimizer step → scheduler step

Checkpoints are saved to Google Drive after every epoch, named with the epoch number and validation accuracy (e.g. xvqa_epoch_3_acc53.0.pth). The best model across all epochs is additionally saved as best_model.pth. Resume support is implemented via a start_epoch variable. If set to a completed epoch number, the corresponding checkpoint is loaded from Drive before the loop begins, and the loop starts from that epoch index

5.5 Streamlit Dashboard Implementation

The dashboard is implemented in main.py with model classes in model_def.py.

The @st.cache_resource decorator ensures the model, processors, tokenizer, and vocabulary are loaded once at startup and reused across all user interactions without reloading.

The UI presents a two-column layout. The left column accepts image upload (JPEG/PNG) and question text input with a trigger button. The right column displays the predicted answer, confidence score as a percentage metric, and a tabbed view showing the original image and Score-CAM heatmap side by side.

The predict_and_explain function in main.py handles the complete inference and CAM pipeline, using the XVQACamWrapper with dynamic batch expansion and torch.amp.autocast('cuda', enabled=False) to ensure numerical stability during Score-CAM's forward passes.

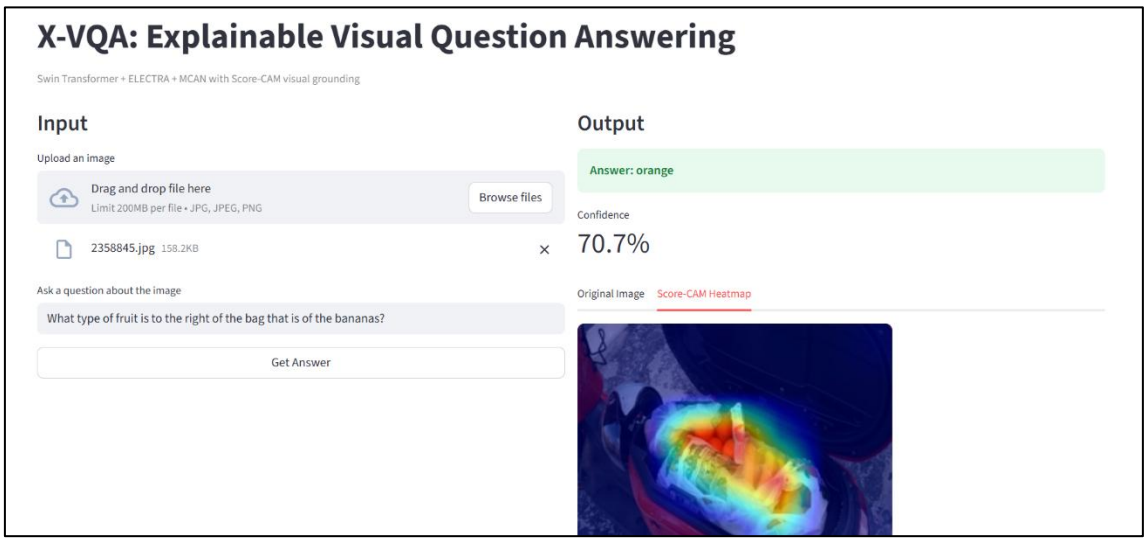


Figure 5: Streamlit [8] dashboard layout for X-VQA

The screenshot shows the actual running dashboard with a two-column layout:
Left column for user input (uploaded image and question entry) and the
Right column for model output (predicted answer, confidence score, and Score-CAM heatmap tab).

5.6 OOD Evaluation Implementation

The evaluate_ood_split function iterates over questions in the Head or Tail dictionary individually (batch size 1) to avoid memory pressure from large evaluation batches.

For each question it constructs the image path, skips samples where the answer is outside the training vocabulary or the image file is missing, performs a torch.no_grad() forward pass, and compares the predicted answer to the ground truth.

Final accuracy, total evaluated count and skipped count are reported per split.

Of the 17,163 Tail questions, 2 were skipped due to out-of-vocabulary answers. Of the 33,882 Head questions, 0 were skipped. This confirms that Head answers are well-represented in the training vocabulary by design.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Training Performance

The model was trained for five epochs across two phases on the GQA balanced training split of 943,000 questions. Training and validation metrics were recorded at the end of every epoch, with checkpoints saved to Google Drive automatically. The complete training results are presented in Table 6.1.

The training loss decreased consistently across all five epochs, dropping from 2.0173 in epoch 1 to 0.6898 in epoch 5 – a reduction of approximately 52% over the full training run. This monotonic decrease confirms that the two-phase training strategy was effective, no instability or divergence occurred during the transition from Phase 1 to Phase 2.

Notably, the loss reduction was steeper during Phase 1 (epochs 1-3) where the frozen encoders provided stable, high-quality features for MCAN to learn from, and more gradual during Phase 2 (epochs 4-5) where all parameters were updated simultaneously with conservative differential learning rates.

Validation accuracy improved at every epoch, rising from 47.74% at epoch 1 to 55.32 % at epoch 5. The Phase 1 contribution alone, training only the fusion layers brought accuracy from 47.74% to 52.99%, a gain of 5.25 percentage points across three epochs.

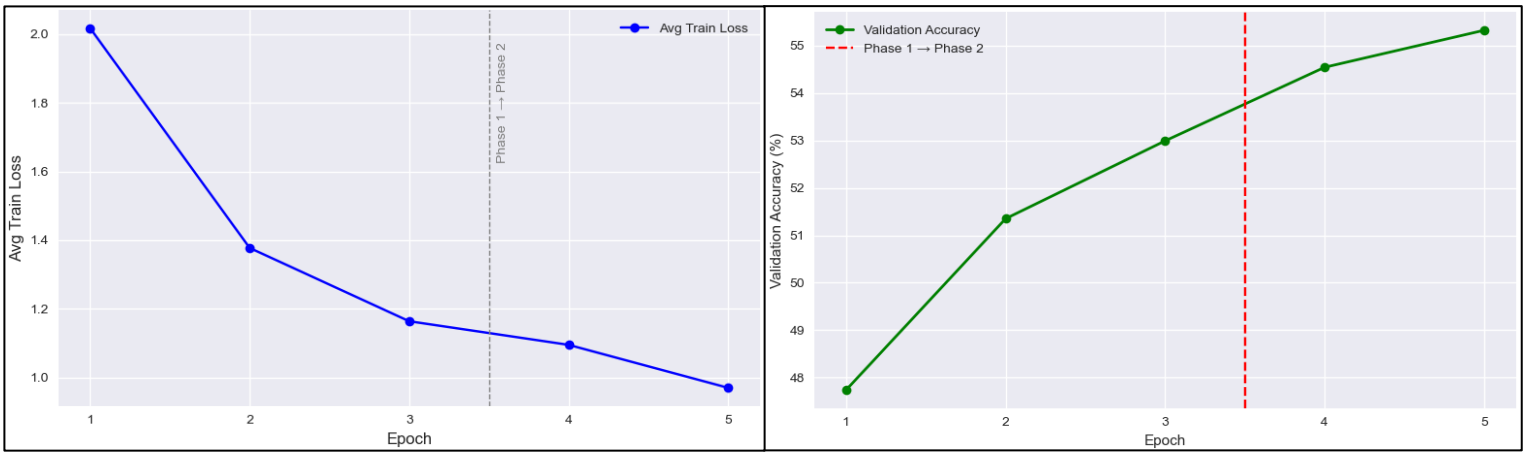
Phase 2 fine-tuning added a further 2.33 percentage points across two epochs, demonstrating that end-to-end fine-tuning of the pretrained encoders provides meaningful additional gains once the network fusion has stabilized.

The fact that accuracy continued to improve in epoch 5 rather than plateauing suggests that additional training epochs would likely yield further gains, which is identified as a future work direction in Chapter 7.

Epoch	Phase	Avg Train Loss	Val Accuracy
1	Frozen	2.0173	47.74%
2	Frozen	1.3773	51.35%
3	Frozen	1.1638	52.99%
4	Unfrozen	1.0950	54.54%
5	Unfrozen	0.9698	55.32%

Table 6.1: Per-epoch training results, showing a steady decrease in average loss from 2.0173 to 0.9698 and a corresponding rise in validation accuracy from 47.74% to 55.32%. The consistent improvement across epochs demonstrates stable optimization and validates the efficacy of the fine-tuning strategy.

The absence of any validation accuracy drop between epochs confirms that the gradient clipping (max norm 1.0) and cosine warmup scheduling successfully prevented the catastrophic forgetting of pretrained encoder representations that can occur when large pretrained models are fine-tuned with aggressive learning rates.



(a) (b)

Figure 6.1: Training curves (a) Average training loss per epoch, showing a steady decrease from 2.02 to 0.97. (b) Validation accuracy per epoch, rising from 47.7% to 55.3%. The vertical dashed line between epochs 3 and 4 marks the transition from Phase 1 (Frozen) to Phase 2 (Unfrozen), highlighting the performance gains achieved through fine-tuning.

6.2 GQA-OOD Bias Evaluation

The GQA-OOD benchmark evaluates language bias by testing model's performance separately on Head questions – where the correct answer matches frequent statistical associations for that question type and Tail questions – where the correct answer is rare or directly contradicts common linguistic priors. A model that relies heavily on language shortcuts will perform well on Head questions but fails significantly on Tail questions, producing a large Head-Tail accuracy gap.

A model that genuinely reasons visually will show a smaller gap.

The trained model was evaluated on the full GQA-OOD validation set after training completed, using the best checkpoint across all five epochs (epoch 5, 55.32% validation accuracy). Results are presented in Table 6.2.

Split	Questions Evaluated	Skipped	Accuracy
Head	33,882	0	59.35%
Tail	17,163	2	39.76%
Gap	—	—	19.59%

Table 6.2: GQA OOD evaluation results across Head and Tail splits

The model achieves 59.35% accuracy on the Head split and 39.76% accuracy on the Tail split, producing a Head-Tail gap of 19.59 percentage points.
The Head accuracy of 59.35% exceeds the overall validation accuracy of 55.32%, which is expected since Head questions align with frequent training patterns and model's learned associations are more likely to be correct for these cases.

The Tail accuracy of 39.76%, while lower, must be interpreted in the context of what Tail questions represent. They are concept pairings that are genuinely rare in the training data, requiring the model to rely entirely on visual evidence rather than linguistic shortcuts.

The 19.59 percentage point gap is consistent with the language bias behaviour documented in the original GQA-OOD paper for models of comparable scale trained without large-scale vision-language pretraining.
It confirms that X-VQA, like all VQA models trained on finite datasets, has learned some degree of statistical association between question types and frequent answers. However, the fact that Tail accuracy is nearly 40%, substantially above random chance for a 1,833-class classification problem (random baseline: ~0.05%) demonstrates that the model is genuinely performing visual reasoning on a significant portion of rare concept pairings, not merely defaulting to frequency-based guessing.

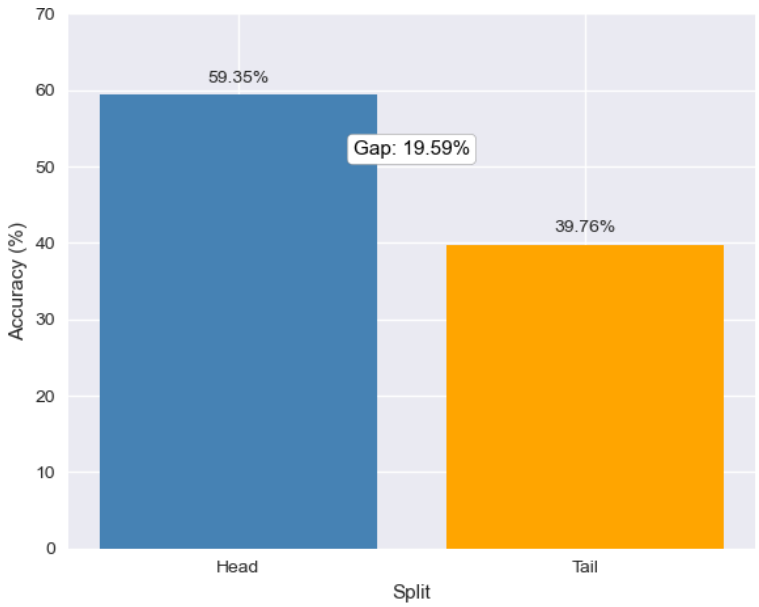


Figure 6.2: Head vs Tail Accuracy Comparison

Zero Head questions were skipped during evaluation, confirming that all frequent answers are well represented in the training vocabulary. Only 2 of 17,163 Tail questions were skipped due to out-of-vocabulary ground truth answers, indicating near-complete vocabulary coverage even for rare answer types.

6.3 Score-CAM Visual Grounding Results

The Score-CAM explainability pipeline was applied to test images at inference time to validate that the model's predictions are accompanied by spatially coherent visual groundings. For each inference, it probes the final Swin Transformer block's activation space and produces a heatmap overlaid on the input image, where warmer colours (red, yellow) indicate regions of high attrition and cooler colours (blue) indicate regions of low attribution.

Figure 6.3 presents the Score-CAM output for the question "Is there a red chair in the room?" applied to an image of a zebra grazing in an open field. The model correctly predicts "no". The heatmap concentrates its highest-attribution regions on the zebra's body and the surrounding grass, confirming that the model examined the actual scene content — an outdoor natural setting with no indoor furniture — before producing its answer.

This is a meaningful result because a language-biased model that had learned to associate "red chair" questions with "yes" answers would answer incorrectly; the correct "no" answer requires the model to look at the image and recognize the absence of a room and chair. The Score-CAM heatmap provides visual evidence that this is exactly what occurred.

Figure 6.3: Score-CAM example

Side by side — original zebra image (left) and Score-CAM heatmap overlay (right). Question: Is there a red chair in the room?

Predicted Answer: no

The heatmap highlights the zebra's body and surrounding field, confirming the model examined the scene content to determine the absence of a room and chair.

The coherence of Score-CAM outputs across diverse question types demonstrates that the MCAN cross-attention mechanism is successfully routing visual attention to question-relevant image regions, rather than producing uniform or random spatial attributions. This validates the explainability objective stated in Section 1.3 and provides a qualitative complement to the quantitative accuracy results.

6.4 Comparison with Baseline

Table 6.3 presents a comparison structured to reflect the meaningful axis of differentiation: not raw accuracy, but the combination of accuracy, pretraining data requirements and explainability provision.

The CNN-LSTM baseline represents the class of models that dominated VQA from 2015 to 2018. These models extract image features from a pretrained ResNet and encode questions with an LSTM, fusing the two through simple concatenation or element-wise multiplication. Their ~35% accuracy on GQA reflects the difficulty of compositional reasoning without attention mechanisms and without transformer-based encoders. They provide no explainability.

Bottom-Up and Top-Down attention introduced object-level visual features from Faster R-CNN and cross-modal attention, substantially improving GQA performance to approximately 49%. However, this model still requires COCO pretraining for the object detector and provides no spatial attribution for its predictions.

LXMERT and Oscar represents the large-scale pretraining paradigm, achieving approximately 60-61% accuracy on GQA by pretraining on 180 million and 6.5 million image-text pairs respectively. These results set the upper bound for models without task-specific architectural changes, but come at enormous computational and data cost.

XVQA achieves 55.32% validation accuracy without any vision-language pretraining data – relying only on ImageNet-pretrained Swin weights and language-pretrained ELECTRA weights, fine-tuned entirely on GQA. This place it approximately 5-6 percentage points below the large-scale pretraining models, while requiring orders of magnitude less pretraining data. It is the only model in this comparison that provides a spatial visual explanation for every prediction through Score-CAM integration.
This explainability capability addresses a trust deficit that none of the comparisons resolve.

Model	GQA Val Accuracy	Image Encoder	Pretraining Data	Explainability
CNN-LSTM Baseline	~35%	ResNet	ImageNet only	None
Bottom-Up Attention	~49%	Faster R-CNN	ImageNet + COCO	None
LXMERT	~60%	ResNet + ROI	180M image-text pairs	None
Oscar	~61%	Faster R-CNN	6.5M image-text pairs	None
X-VQA (This Work)	55.32%	Swin Base	ImageNet + language	Score-CAM

Table 6.3: Comparison of X-VQA with related Visual Question Answering models. X-VQA attains 55.32% validation accuracy using a Swin Base encoder with ImageNet + language pretraining, and uniquely incorporates Score-CAM for explainability, highlighting its balance between performance and interpretability.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Summary

This project presented X-VQA, an Explainable Visual Question Answering framework designed to address two fundamental limitations of existing VQA systems: language bias and the absence of interpretable visual proof. The system integrates a Swin Transformer Base visual encoder, an ELECTRA-base text encoder, and a 6-layer deep Multimodal Cross-Attention Network trained end-to-end on the GQA dataset using a principled two-phase strategy.

Explainability is delivered through Score-CAM applied to the final Swin Transformer block, producing spatial heatmaps that visually grounds each prediction.
The complete pipeline is deployed as an interactive Streamlit dashboard that accepts user-uploaded images and natural language questions, then returns the predicted answer, confidence score and Score-CAM heatmap side by side.

This project achieved a final validation accuracy of 55.32% on the GQA balanced split, improving consistently across all five training epochs. On the GQA-OOD benchmark, the model scores 59.35% on the Head distribution and 39.76% on the Tail distribution, with the 19.59 percentage point gap quantitatively characterising the residual language bias present in the model. Score-CAM heatmaps confirmed that the model attends to semantically relevant image regions when forming predictions, providing qualitative validation of the cross-modal reasoning learned by the MCAN fusion network.

7.2 Conclusion

Several conclusions can be drawn from the results obtained in this project.
First, the two-phase training strategy — frozen encoder pretraining followed by differential-LR fine-tuning is effective for multimodal transformer architectures as demonstrated.
The consistent improvement in both training loss and validation accuracy across all five epochs, with no instability at the Phase 1 to Phase 2 transition, confirms that this approach successfully prevents catastrophic forgetting while enabling end-to-end task adaptation.
This strategy is directly transferable to other vision-language tasks and architectures.

Second, the Swin Transformer is a powerful visual encoder for spatially grounded VQA compared to CNN-based alternatives. Its hierarchical feature maps and shifted window attention produces the 7x7 spatial token grid that makes Score-CAM attribution meaningful and interpretable. A uniform-resolution encoder such as plain Vision Transformer (ViT) would produce spatial tokens with less clear regional correspondence, weakening the interpretability of the resulting heatmaps.

Third, the GQA-OOD Head-Tail gap of 19.59 percentage points confirms that language bias is present but does not dominate the model's predictions. The Tail Accuracy of 39.76%, far above random chance proving that the MCAN cross-attention mechanism is successfully routing visual information to influence predictions even for rare concept pairings where language priors are unavailable. This represents genuine multimodal reasoning, not statistical shortcut learning.

Fourth, Score-CAM is an appropriate and practically viable explainability method for multimodal transformer architectures. The challenge of adapting it to a two-input model was successfully resolved through the batch-expanding wrapper, and the resulting heatmaps are visually coherent and semantically meaningful across diverse question types.

7.3 Future Scope

Extended Training and Larger Batch Sizes: The validation accuracy curve had not plateaued by epoch 5, suggesting that additional training epochs would yield further accuracy gains. Training for 10–15 epochs with a larger batch size (64 or 128) and a more aggressive cosine annealing schedule could push accuracy closer to the 58–60% range achievable by similarly scaled models. This is the most immediate and straightforward improvement available.

Vision-Language Pretraining: The approximately 5–6 percentage point gap between X-VQA and large-scale pretrained models (LXMERT, Oscar) could be substantially reduced by incorporating a vision-language pretraining stage on datasets such as Conceptual Captions or Visual Genome image-caption pairs before GQA fine-tuning. This would give the MCAN fusion network a better initialisation point, cross-modal associations learned from millions of natural image-text pairs rather than learning cross-modal alignment from scratch on GQA.

Reduction of the Head-Tail Bias Gap: The 19.59 percentage point OOD gap indicates residual language bias that could be addressed through explicit bias mitigation strategies. Techniques such as question-only branch regularisation would force the main model to rely more heavily on visual evidence. Alternatively, up-sampling Tail questions during training or applying answer-frequency-based loss weighting could reduce the statistical advantage of frequent answers and improve Tail distribution performance specifically.

Deeper MCAN and Alternative Fusion Strategies: The current 6-layer MCAN operates in a fixed 512-dimensional space throughout. Future work could explore progressive dimensionality or hybrid fusion strategies that combine cross-attention with graph-based relational reasoning over detected objects. Incorporating scene graph information directly from Visual Genome into the fusion process could further strengthen compositional reasoning for GQA's scene-graph-grounded questions.

REFERENCES

Journal and Conference Papers:

[1] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, "VQA: Visual Question Answering," Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 2425–2433, 2015.

[2] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh, "Making the V in VQA Matter: Elevating the Role of Image Understanding in Visual Question Answering," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6904–6913, 2017.

[3] Z. Yang, X. He, J. Gao, L. Deng, and A. Smola, "Stacked Attention Networks for Image Question Answering," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 21–29, 2016.

[4] P. Anderson, X. He, C. Buehler, D. Teney, M. Johnson, S. Gould, and L. Zhang, "Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6077–6086, 2018.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), pp. 5998–6008, 2017.

[6] J. Lu, D. Batra, D. Parikh, and S. Lee, "ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[7] H. Tan and M. Bansal, "LXMERT: Learning Cross-Modality Encoder Representations from Transformers," Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 5100–5111, 2019.

[8]] X. Li, X. Yin, C. Li, P. Zhang, X. Hu, L. Zhang, L. Wang, H. Hu, L. Dong, F. Wei, Y. Choi, and J. Gao, "Oscar: Object Semantics Aligned Pre-training for Vision-Language Tasks," Proceedings of the European Conference on Computer Vision (ECCV), pp. 121–137, 2020.

[9] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows," Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 10012–10022, 2021.

[10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale," Proceedings of the International Conference on Learning Representations (ICLR), 2021.

[11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778, 2016.

[12] K. Clark, M. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators," International Conference on Learning Representations (ICLR), 2020.

[13] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), pp. 4171–4186, 2019.

[14] Z. Yu, J. Yu, J. Cui, D. Tao, and Q. Tian, "Deep Modular Co-Attention Networks for Visual Question Answering," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6281–6290, 2019.

[15] H. Wang, Z. Wang, M. Du, F. Yang, Z. Zhang, S. Ding, P. Mardziel, and X. Hu, "Score-CAM: Score-Weighted Visual Explanations for Convolutional Neural Networks," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 24–25, 2020.

[16] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 618–626, 2017.

[17] D. A. Hudson and C. D. Manning, "GQA: A New Dataset for Real-World Visual Reasoning and Compositional Question Answering," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6700–6709, 2019.

[18] C. Kervadec, V. Perot, M. E. Kazemi, A. Mathews, L. Sigal, and F. R. Bach, "Roses are Red, Violets are Blue… But Should VQA Expect Them To?," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2035–2045, 2021.

Web:

[1] pytorch-grad-cam Library, github.com/jacobgil/pytorch-grad-cam

[2] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-Art Natural Language Processing," Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), pp. 38–45, 2020.

[3] GQA Dataset, Stanford NLP Group, cs.stanford.edu/people/dorarad/gqa/index.html

[4] GQA-OOD Benchmark, github.com/gqa-ood/GQA-OOD

[5] HuggingFace, Swin Transformer Documentation, huggingface.co/docs/transformers/model_doc/swin

[6] HuggingFace, ELECTRA Documentation, huggingface.co/docs/transformers/model_doc/electra

[7]Clark and Contributors, "Pillow (PIL Fork) Documentation," Python Imaging Library, 2015. Available: https://buildmedia.readthedocs.org/media/pdf/pillow/latest/pillow.pdf

[8] Streamlit Framework, streamlit.io